

Name of the Student	V Sunil Kumar
Reg. No	192425292
GitHub Link	https://github.com/192425292simats-byte/MLA0403-DEEP-LEARNING.git

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 2: Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches. (BTL-4)

CO Weightage : 15%

Assessment Tool Weightage: 40%

Duration: 90 minutes

Marks: 25

Code of Conduct:

I V Sunil Kumar/192425292 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: V. Sunil Kumar

Scenario-Based Questions

CO-AT2

QUESTIONS: 05

Total Marks:25

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

ANSWER:

The hospital can use an Autoencoder-based unsupervised deep learning technique to identify abnormal ECG patterns from thousands of unlabeled ECG records. An autoencoder is suitable because it can learn useful patterns from data without requiring manually labeled examples.

The autoencoder consists of two main parts: an Encoder and a Decoder. The encoder takes the ECG signal as input and converts it into a compact representation called the latent representation. The decoder then reconstructs the original ECG signal from this representation.

Workflow:

1. Collect thousands of unlabeled ECG records from patients.
2. Preprocess the ECG signals by removing noise and normalizing the values.
3. Feed the ECG data into the autoencoder.
4. The encoder compresses the ECG signal into a lower-dimensional representation.
5. The decoder reconstructs the ECG signal.
6. The autoencoder is trained to minimize the difference between the original and reconstructed ECG signals.
7. After training, new ECG records are given to the model.
8. Normal ECG patterns usually have a low reconstruction error because the model has learned their characteristics.
9. Abnormal ECG patterns produce a higher reconstruction error because they differ from the learned patterns.
10. ECG records with errors above a selected threshold can be flagged for further medical examination.

Advantages:

- **Does not require labeled ECG data.**
- Can process large amounts of medical records.
- Automatically learns important ECG features.
- Useful for detecting unusual or abnormal patterns.

Conclusion:

An autoencoder is an effective unsupervised technique for ECG anomaly detection because it learns the underlying structure of ECG signals and identifies records that significantly differ from the learned patterns.

2. Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

ANSWER:

A retail company may have customer purchase data containing hundreds or thousands of features. Using all these features directly for customer segmentation can increase computational complexity and may contain redundant information. An autoencoder can be used for dimensionality reduction and representation learning.

An autoencoder learns a compact representation of the original customer data while attempting to preserve the most important information.

Working:

1. Collect customer information such as purchase frequency, total spending, product preferences, transaction history, and number of purchases.
2. Preprocess the data by handling missing values and normalizing numerical features.
3. Give the high-dimensional customer data to the encoder.
4. The encoder gradually reduces the number of features through its hidden layers.
5. The smallest layer is called the latent or bottleneck layer.
6. This bottleneck contains a compact representation of customer behavior.
7. The decoder reconstructs the original customer data from this compressed representation.
8. During training, the model minimizes the reconstruction error.
9. After training, only the encoder is required for dimensionality reduction.
10. The compressed representations can be given to a clustering algorithm such as K-Means.
11. Customers with similar representations can be grouped into different segments.
12. These segments can be used for targeted marketing and personalized offers.

Example:

A dataset containing 100 customer features can be compressed into a 10-dimensional latent representation. The 10 features can then be used for customer segmentation instead of processing all 100 original features.

Advantages:

- Reduces dimensionality.
- Removes redundant information.
- Automatically learns important features.
- Makes clustering faster.
- Provides useful customer representations.

Conclusion:

Autoencoders provide an effective method of representation learning because they transform high-dimensional purchase data into a smaller and meaningful representation that can be used for customer segmentation.

3. Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural

network. Recommend architectural changes involving network depth and activation functions.

ANSWER:

If a facial recognition system performs poorly with a shallow neural network, the architecture can be improved by increasing the depth and width of the network and selecting appropriate activation functions.

A shallow network has only a small number of hidden layers and may not be able to learn complex facial patterns. Facial recognition requires learning multiple levels of features, from simple edges to complex facial structures.

Recommended architectural changes:

1. Increase network depth:
Add more hidden layers so that the network can learn increasingly complex facial features.
2. Increase network width:
Increase the number of neurons in some hidden layers so that the network can learn more features from facial images.
3. Use ReLU activation:
ReLU can be used in hidden layers because it introduces non-linearity and allows the network to learn complex relationships efficiently.
4. Use Softmax for classification:
The output layer can use Softmax when the system needs to identify a person from multiple known classes.
5. Use suitable regularization:
Techniques such as dropout can reduce overfitting when the network becomes deeper.
6. Use normalization:
Batch normalization can help stabilize and speed up training.
7. Train with sufficient data:
The deeper network should be trained using a sufficiently large and diverse collection of facial images.

Feature learning process:

Input Face Image → Edges → Shapes → Eyes/Nose/Mouth → Facial Features → Complete Face Representation → Person Classification

A deeper network can therefore learn hierarchical representations that a shallow network may fail to capture.

Advantages:

- Learns complex facial features.
- Improves recognition accuracy.
- Extracts hierarchical representations.
- ReLU helps efficient learning.
- Softmax provides multi-class identification.

Conclusion:

Increasing the depth and appropriate width of the network, together with suitable activation functions such as ReLU and Softmax, can improve the ability of the facial recognition system to learn complex facial patterns.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

ANSWER:

A streaming service can use a Restricted Boltzmann Machine (RBM) to build a personalized movie recommendation system. RBMs are generative neural networks that can learn hidden patterns from data without requiring explicitly labeled training data.

An RBM consists of two layers:

- Visible layer: Represents the input data, such as movies watched or rated by users.
 - Hidden layer: Learns hidden relationships and preferences from the viewing data.
- There are connections between the visible and hidden layers, but there are no connections between neurons within the same layer.

Workflow:

1. Collect user viewing history from the streaming platform.
2. Represent movies using values such as watched/not watched or rating values.
3. Provide this information to the visible layer of the RBM.
4. The hidden layer learns patterns in user preferences.
5. During training, the RBM adjusts its weights to learn relationships between movies.
6. For example, if users who watch Movie A and Movie B frequently also watch Movie C, the RBM can learn this hidden relationship.
7. When a user watches or rates certain movies, the RBM predicts preferences for other movies.
8. Movies with high predicted preference values are selected as recommendations.
9. The system can continuously update recommendations as the user's viewing history changes.

Example:

If a user frequently watches action and science-fiction movies, the RBM can learn this preference pattern and recommend other movies that match similar hidden features.

Advantages:

- Works with unlabeled user behavior data.
- Learns hidden preference patterns.
- Provides personalized recommendations.
- Can handle large user-movie datasets.
- Useful for discovering relationships between movies.

Conclusion:

RBMs can automatically learn hidden patterns from viewing histories and use those patterns to predict movies that a user may prefer, making them useful for recommendation engines.

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

ANSWER:

The manufacturing company can develop a deep learning-based defect detection system using sensor readings collected every second. The system can automatically learn useful representations from sensor data and classify products as defective or non-defective.

Sensors may collect information such as temperature, pressure, vibration, humidity, speed, current, and machine operating conditions.

Proposed Workflow:

Sensor Data → Data Preprocessing → Feature/Representation Learning → Deep Neural Network → Classification → Defective / Non-Defective

Steps:

1. Data Collection:
Collect sensor readings continuously from machines during the manufacturing process.
2. Data Preprocessing:
Remove incorrect or missing readings and normalize the sensor values so that different measurements can be processed effectively.
3. Representation Learning:
A deep neural network automatically learns important patterns from the raw sensor data instead of requiring engineers to manually select every feature.
4. Hidden Layers:
Multiple hidden layers learn increasingly complex relationships between sensor measurements.
5. Training:
Historical sensor readings can be used to train the model. If defective and non-defective examples are available, the model can learn to distinguish between them.
6. Classification:
The final layer produces the classification result, such as:
 - 0 – Non-Defective
 - 1 – Defective
7. Real-Time Prediction:
When new sensor readings arrive every second, the trained model analyzes them and predicts the condition of the product.
8. Alert Generation:
If the model detects a high probability of a defect, the system can generate an alert for the manufacturing team.

Example:

Suppose a machine normally operates with a specific range of temperature and vibration. If the vibration suddenly increases while the temperature and pressure also show unusual patterns, the deep learning model can recognize this combination as a possible defective-product pattern.

Benefits:

- Automatic feature learning.

- Real-time defect detection.
- Reduces manual inspection.
- Improves product quality.
- Reduces manufacturing waste.
- Helps identify unusual sensor patterns.
- Can support predictive maintenance.

Conclusion:

A deep learning solution based on representation learning can automatically learn complex relationships from continuous sensor data and classify products based on their quality. This allows the manufacturing company to detect defects quickly, improve production efficiency, and reduce operational costs.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts

Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand